Algoritmo de Quine-McCluskey em C++ | Parte prática apenas

***Resumo*—Este artigo documenta a implementação prática do algoritmo de Quine-McCluskey em C++, um minimizador de funções booleanas baseado em leitura de arquivos no formato PLA. São apresentados a estrutura do programa, o formato de entrada e saída adotado, e os resultados obtidos em três casos de teste.**

I. Introdução

A minimização de funções booleanas é um problema central no projeto de circuitos digitais. O algoritmo de Quine-McCluskey [**?**] resolve esse problema de forma sistemática para qualquer número de variáveis. A implementação desenvolvida recebe um arquivo PLA como entrada, executa a minimização e gera um arquivo PLA de saída com a expressão booleana simplificada [**?**].

II. Estrutura do Programa

O programa é executado via linha de comando, recebendo o caminho do arquivo de entrada como argumento:

```
./src input/arquivo.pla
```

Os arquivos de entrada ficam na pasta `input/` e o resultado é salvo automaticamente em `output/result.pla`. O fluxo interno segue três etapas: (1) leitura do PLA e extração dos mintermos com saída 1; (2) geração dos implicantes primos por combinações sucessivas; (3) seleção da cobertura mínima.

III. Formato PLA

A diretiva `.i` indica o número de entradas e `.o` o número de saídas. Cada linha de dados contém os bits de entrada, um espaço e o valor da saída. O símbolo `-` representa *don't care* e `.e` encerra o arquivo.

IV. Resultados

A. Caso 1: Função AND de 2 variáveis

Entrada (`input/and.pla`):

```
.i 2
.o 1
11 1
.e
```

Saída (`output/result.pla`):

```
F = AB

PLA output:
.i 2
.o 1
11 1
.e
```

A função AND só vale 1 quando ambas as entradas são 1. Como há apenas um mintermo, nenhuma simplificação é possível e o programa retorna $F = AB$.

B. Caso 2: Função de 3 variáveis

Entrada (`input/ex01_funcao3var.pla`):

```
.i 3
.o 1
0-0 1
-11 1
.e
```

Saída (`output/result.pla`):

```
F = A'C' + BC

PLA output:
.i 3
.o 1
0-0 1
-11 1
.e
```

O implicante `0-0` representa $A'C'$, com B irrelevante, e `-11` representa BC, com A irrelevante.

C. Caso 3: Função de 4 variáveis

Entrada (`input/ex00_f_da_folha.pla`):

```
.i 4
.o 1
0-00 1
-000 1
.e
```

Saída (`output/result.pla`):

```
F = A'C'D' + B'C'D'

PLA output:
.i 4
.o 1
0-00 1
-000 1
.e
```

O implicante `0-00` representa $A'C'D'$ e `-000` representa $B'C'D'$, cobrindo exatamente os mintermos 0, 4 e 8.

V. Código Fonte

O código fonte completo da implementação está disponível publicamente em:

https://github.com/LuccaDultra/Relatorios-SD/tree/main/PSD/Ciclo%202/Lab%206/qcluskey-algorithm

VI. Conclusão

A implementação mostrou-se correta nos três casos testados, produzindo expressões mínimas coerentes com os mintermos fornecidos. O uso do formato PLA tanto na entrada quanto na saída facilita a integração com outras ferramentas de síntese lógica.

Referências

[1] E. J. McCluskey, "Minimization of Boolean Functions," *Bell System Technical Journal*, vol. 35, no. 6, pp. 1417–1444, 1956.

[2] B. Stroustrup, *The C++ Programming Language*, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2013.